



COMSATS University Islamabad, Wah Campus

Department of Computer Science

SQE Project Report

BSE-6A

Course Title:

Software Quality Engineering

Course Instructor:

Dr. Wasif Nisar

Submitted by:

Name	Reg.No
Muhammad Sami	SP23-BSE-039
Ahmer Abbas	SP23-BSE-023
Mustafa Riaz	SP23-BSE-035
Naveed Umer	SP23-BSE-026

Contents

1	INTRODUCTION	6
1.1	System Introduction.....	6
1.2	Background of the System.....	6
1.3	Objectives of the System	6
1.4	Significance of the System	6
2	REQUIREMENT SPECIFICATIONS	8
2.1	Product Scope.....	8
2.2	Product Description	8
2.2.1	Product Perspective.....	8
2.2.2	Product Functionality	8
2.2.3	Users and Characteristics	9
2.2.4	Operating Environment.....	9
2.3	Specific Requirements.....	9
2.3.1	Functional Requirements.....	9
2.3.2	Behavioral Requirements	10
2.3.3	External Interface Requirements	13
2.4	Non-functional Requirements.....	14
2.4.1	Performance Requirements	14
2.4.2	Safety and Security Requirements	14
2.4.3	Software Quality Attributes	15
3	DESIGN SPECIFICATIONS	17
3.1	Introduction	17
3.2	Composite Viewpoint.....	17
3.3	Logical Viewpoint.....	17
3.4	Information Viewpoint	19
3.5	Interaction Viewpoint.....	19
3.6	State Dynamics Viewpoint.....	22
3.7	Algorithmic Viewpoint.....	23
4	DEVELOPMENT AND TOOLS.....	27
4.1	Introduction	27
4.2	Development Plan	27
4.3	Development Tools	27
4.4	Conclusion and Future Work/Extensions	28
5	QUALITY ASSURANCE	29

5.1	Introduction	29
5.2	Traceability Matrix	29
5.3	Test Plan	31
6	USER MANUAL	35
6.1	Introduction	35
6.2	Hardware/Software Requirements for the System	35
6.3	Installation guide for Application.....	35
6.4	Operating Manual	36
6.4.1	Starting the Application	36
6.4.2	User Operations:	37
6.4.3	Organizer Operations:	38
6.4.4	Admin Dashboard:.....	38

List of Tables

Table 2.1 Fully Dressed Use Case	11
Table 2.2 Fully Dressed Use Case	11
Table 2.3 Fully Dressed Use Case	11
Table 2.4 Fully Dressed Use Case	12
Table 2.5 Fully Dressed Use Case	12
Table 2.6 Fully Dressed Use Case	12
Table 2.7 Fully Dressed Use Case	13
Table 4.1 Work Plan division	27
Table 5.1 Traceability Matrix	30
Table 5.2 Test Case User Registration.....	31
Table 5.3 Test Case User Login.....	31
Table 5.4 Test Case Create Event	32
Table 5.5 Test Case Approve Event.....	32
Table 5.6 Test Case Booking Event.....	33
Table 5.7 Test Case Make Payment.....	33
Table 5.8 Test Case View Booking Status.....	34
Table 5.9 Test Case Edit Event.....	34
Table 5.10 Test Case View Reports.....	34

List of Figures

Figure 2.1 Use Case Diagram of Event Management System	13
Figure 3.1 Deployment Diagram of Event Management	17
Figure 3.2 Class Diagram of Event Management	18
Figure 3.3 Sequence Diagram of Event Management	20
Figure 3.4 Sequence Diagram of Event Management	20
Figure 3.5 Sequence Diagram of Event Management	21
Figure 3.6 Sequence Diagram of Event Management	21
Figure 3.7 Sequence Diagram of Event Management	22
Figure 3.8 State Machine Diagram of Event Management.....	23
Figure 6.1 Home Page of Event Management	36
Figure 6.2 Login Page of Event Management	37
Figure 6.3 User Dashboard of Event Management.....	37
Figure 6.4 Organizer Dashboard of Event Management	38
Figure 6.5 Admin Dashboard of Event Management	39

1 INTRODUCTION

1.1 System Introduction

EventHub is a web-based event management and ticket booking system developed to simplify the process of organizing and attending events. The system allows users to browse events, book tickets, and manage their bookings through a secure online platform. Organizers can create events, manage attendees, and track revenue, while administrators control system operations. The main purpose of EventHub is to provide a centralized, secure, and efficient solution for managing all event-related activities. It improves user experience by automating events and booking processes.

1.2 Background of the System

In many traditional event management systems, event handling is done manually using paper-based tickets, spreadsheets, or basic online tools. These methods often lead to errors, poor record management, and lack of security. Although some online event platforms exist, many of them do not provide proper role-based access, analytics, or administrative control. EventHub addresses these limitations by offering a complete role-based system with secure authentication, event analytics, and centralized management. This makes the system more reliable and efficient than existing solutions.

1.3 Objectives of the System

The main objective of the EventHub system is to provide an efficient and secure platform for managing events and ticket bookings. It aims to simplify event creation and booking for users and organizers while ensuring data accuracy and security. Another objective is to support role-based access so that each user performs only authorized actions. The system also focuses on improving event visibility and tracking through analytics and reports. Overall, EventHub is designed to reduce manual effort and improve event management efficiency.

1.4 Significance of the System

The EventHub system is significant because it modernizes the event management process by replacing manual and inefficient methods with an automated solution. It helps users easily discover and attend events while enabling organizers to manage operations effectively. The system is useful

for various application areas such as educational seminars, conferences, workshops, concerts, and community events. By providing security, scalability, and ease of use, EventHub enhances trust and transparency in event handling. This makes it a valuable system for both event participants and administrators.

2 REQUIREMENT SPECIFICATIONS

2.1 Product Scope

The EventHub system is designed to provide a completely web-based solution for event management and ticket booking. The system allows users to browse events, book tickets, manage bookings, and receive notifications. Organizers can create and manage events, track attendees, and monitor revenue, while administrators can control users, events, and system settings. The scope of this system does not include mobile applications or third-party payment gateway integration at this stage. The system focuses only on online event handling through a web browser.

2.2 Product Description

2.2.1 Product Perspective

EventHub is a new and self-contained web application developed to replace manual and partially automated event management systems. It operates as a centralized platform where users, organizers, and administrators interact through a browser-based interface. The system follows a client-server architecture where the frontend communicates with a backend server using RESTful APIs, and data is stored in a centralized database. EventHub interacts with external environments such as web browsers and email services. A simple system diagram shows users accessing the frontend, which connects to the backend and database.

2.2.2 Product Functionality

The major functionalities of the EventHub system include:

- User registration and secure login
- Event browsing, searching, and filtering
- Ticket booking and booking management
- Event creation and management by organizers
- Attendee and payment tracking
- Administrative control over users and events
- Analytics and reporting

2.2.3 Users and Characteristics

The EventHub system is used by different types of users based on roles and access levels. General users access the system to browse events and book tickets and usually have basic technical knowledge. Organizers use the system more frequently to create events, manage bookings, and view analytics, requiring moderate technical skills. Administrators are the most important users as they manage the entire system, including users, events, and security, and require advanced system knowledge. Guest users have limited access and mainly browse events without authentication.

2.2.4 Operating Environment

The EventHub system operates in a web-based environment and is accessed through modern web browsers such as Chrome, Firefox, and Edge. The backend runs on a server using Node.js, while data is stored in a Microsoft SQL Server database. The system can operate on Windows or Linux-based servers with stable internet connectivity. Minimum requirements include a standard web browser, internet access, and a server with sufficient memory and processing power. The system peacefully coexists with other web applications running on the same environment.

2.3 Specific Requirements

2.3.1 Functional Requirements

The functional requirements of the EventHub system define the core features and operations that the system must perform to support event management and ticket booking. These requirements describe how users, organizers, and administrators interact with the system and how the system processes their actions. The functional requirements are grouped into major functional areas for clarity.

1. User Management:
 - Users can register and log in securely
 - Users can update their profile (name, email, etc.)
 - The system assigns roles:
 - User
 - Organizer

- Admin
- 2. Event Management:
 - Organizers can create, update, and delete events
 - Users can view event details and check availability
 - Admins can approve or reject events before they are published
- 3. Ticket Booking:
 - Users can book tickets for events
 - The system checks if tickets are available before booking
 - Users can view and manage their bookings
- 4. Payment Handling:
 - The system automatically calculates the total cost
 - The system tracks payment status (paid, pending, rejected)
 - Organizers can confirm or reject payments
- 5. Administrative Functions:
 - Admins can manage users and assign roles
 - Admins can manage event categories
 - Admins can view reports and analytics for the whole system

2.3.2 Behavioral Requirements

1. Actors Description:

Admin: Responsible for approving events, managing users, viewing reports, and monitoring overall system activity.

Organizer: Responsible for creating and editing events and viewing user-related information.

User: Responsible for registering in the system, viewing events, and receiving notifications.
2. Major Use Case Descriptions:

Field	Description
Use Case ID	UC-01
Use Case Name	Register

Primary Actor	User
Description	Users create an account by providing required information.
Preconditions	User is not registered in the system.
Main Flow	User enters details → System validates → Account created.
Postconditions	User account is successfully registered.

Table 2.1 Fully Dressed Use Case

Field	Description
Use Case ID	UC-02
Use Case Name	View Event
Primary Actor	User
Description	User views available events and their details.
Preconditions	Events exist in the system.
Main Flow	User selects event → System displays event details.
Postconditions	Event information is shown to the user.

Table 2.2 Fully Dressed Use Case

Field	Description
Use Case ID	UC-03
Use Case Name	Create Event
Primary Actor	Organizer
Description	Organizers create a new event by entering event details.
Preconditions	Organizer is logged into the system.
Main Flow	Organizer enters details → System saves event.
Postconditions	Events are created and marked for approval.

Table 2.3 Fully Dressed Use Case

Field	Description
Use Case ID	UC-04
Use Case Name	Edit Event

Primary Actor	Organizer
Description	Organizer updates existing event information.
Preconditions	Event already exists.
Main Flow	Organizer edits details → System updates event.
Postconditions	Event details are updated.

Table 2.4 Fully Dressed Use Case

Field	Description
Use Case ID	UC-05
Use Case Name	Approve Event
Primary Actor	Admin
Description	Admin reviews and approves an event.
Preconditions	Event is pending approval.
Main Flow	Admin approves event → System publishes event.
Postconditions	Event becomes visible to users.

Table 2.5 Fully Dressed Use Case

Field	Description
Use Case ID	UC-06
Use Case Name	Manage Users
Primary Actor	Admin
Description	Admin manages user accounts and roles.
Preconditions	Users exist in the system.
Main Flow	Admin updates user data → System saves changes.
Postconditions	User information is updated.

Table 2.6 Fully Dressed Use Case

Field	Description
Use Case ID	UC-07
Use Case Name	View Reports
Primary Actor	Admin

Description	Admin views system reports and summaries.
Preconditions	System data is available.
Main Flow	Admin requests report → System displays report.
Postconditions	Reports are shown to admin.

Table 2.7 Fully Dressed Use Case

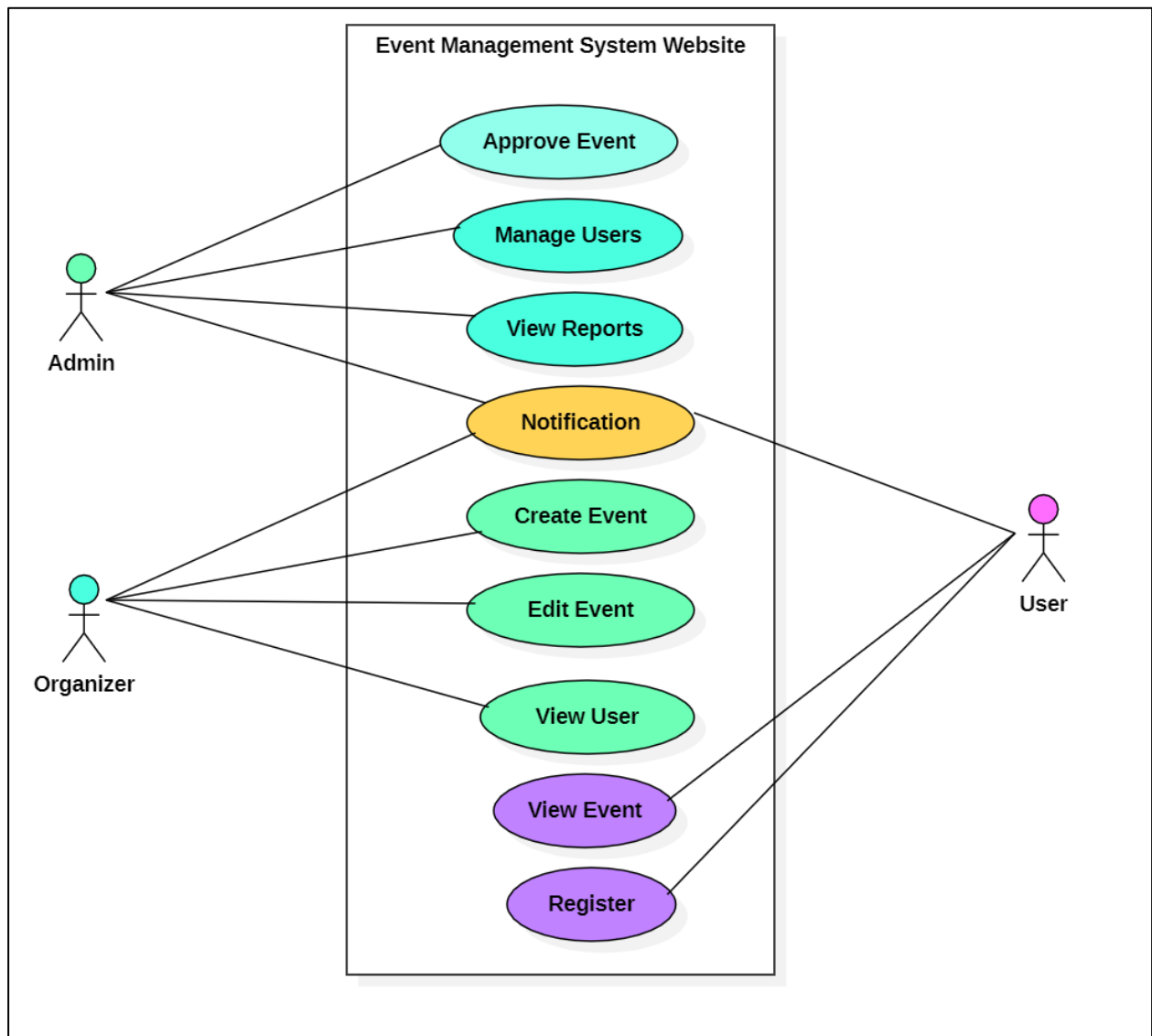


Figure 2.1 Use Case Diagram of Event Management System

2.3.3 External Interface Requirements

2.3.3.1 User Interface

The Event Management System provides a web-based user interface that allows users to interact with the system through a browser. The main screens include the home page, user

registration and login screens, event listing and event detail pages, and a user dashboard. Organizers are provided with interfaces for creating and editing events, while administrators have dashboards for approving events, managing users, and viewing reports. Standard buttons such as submit, cancel, edit, and logout are available on relevant screens, and clear error messages are displayed when invalid input or system errors occur. The interface is designed to be simple, consistent, and easy to use for all user roles.

2.3.3.2 *Other Interfaces (if any)*

The Event Management System does not require any specialized hardware interfaces, as it operates entirely in a web-based environment. The system runs on localhost during development and communicates through standard web browsers using basic HTTP communication. No external hardware devices, third-party systems, or additional software interfaces are involved at this stage of the project.

2.4 Non-functional Requirements

2.4.1 Performance Requirements

The Event Management System performs efficiently under different operating conditions. User actions such as login, event browsing, and ticket booking are completed within a few seconds. Any transaction, including booking and event creation, is processed within **5 to 10 seconds** depending on network conditions. The system supports multiple users accessing the application simultaneously without noticeable delay. Database queries return results promptly, ensuring smooth navigation and consistent system responsiveness.

2.4.2 Safety and Security Requirements

The Event Management System is designed to protect user data and prevent unauthorized access or misuse of system resources. User information is handled securely to avoid data loss, data leakage, or system damage. Access to system features is controlled through authentication and role-based authorization to ensure that only permitted users perform sensitive actions. The system protects user privacy and maintains data integrity during normal operation.

Major safety and security requirements include:

- Secure user authentication using unique login credentials

- Role-based access control for users, organizers, and administrators
- Protection of sensitive users and event data from unauthorized access
- Secure session handling to prevent unauthorized session usage
- Input validation to prevent invalid or harmful system operations

2.4.3 Software Quality Attributes

The Event Management System is developed with a focus on key software quality attributes that improve user experience and simplify system maintenance. These attributes ensure that the system remains reliable, easy to use, adaptable, and maintainable as system requirements grow or change.

1. Reliability:

The system operates consistently without unexpected failures by handling errors properly and validating user input. Core functionalities such as login, event creation, and booking remain stable during normal usage.

2. Usability:

The system provides a simple and intuitive interface that allows users to complete tasks easily with minimal learning effort. Clear navigation and readable messages improve overall user interaction.

3. Maintainability:

The system uses a modular design, separating frontend, backend, and database logic. This structure makes it easy to fix bugs, update features, or modify components without affecting the entire system.

4. Scalability:

The system supports an increasing number of users and events by using server-based architecture and optimized database queries. Performance remains stable as usage increases.

5. Portability:

The system runs on different operating systems through standard web browsers without requiring platform-specific changes.

6. Adaptability (Design for Change):

The system architecture allows new features, such as payment integration or mobile support, to be added with minimal structural changes.

3 DESIGN SPECIFICATIONS

3.1 Introduction

This chapter presents the architectural and detailed design of the Event Management System. It explains how the system components are organized and how they interact with each other. UML diagrams are used to represent different design viewpoints. The chapter helps in understanding the system before implementation and testing.

3.2 Composite Viewpoint

The composite viewpoint identifies the major subsystems of the Event Management System and their deployment. The system is divided into frontend, backend, and database components. A deployment diagram is used to show how users access the system through a web browser, how requests are handled by the backend server, and how data is stored in the database. This viewpoint provides a physical and logical overview of the complete system.

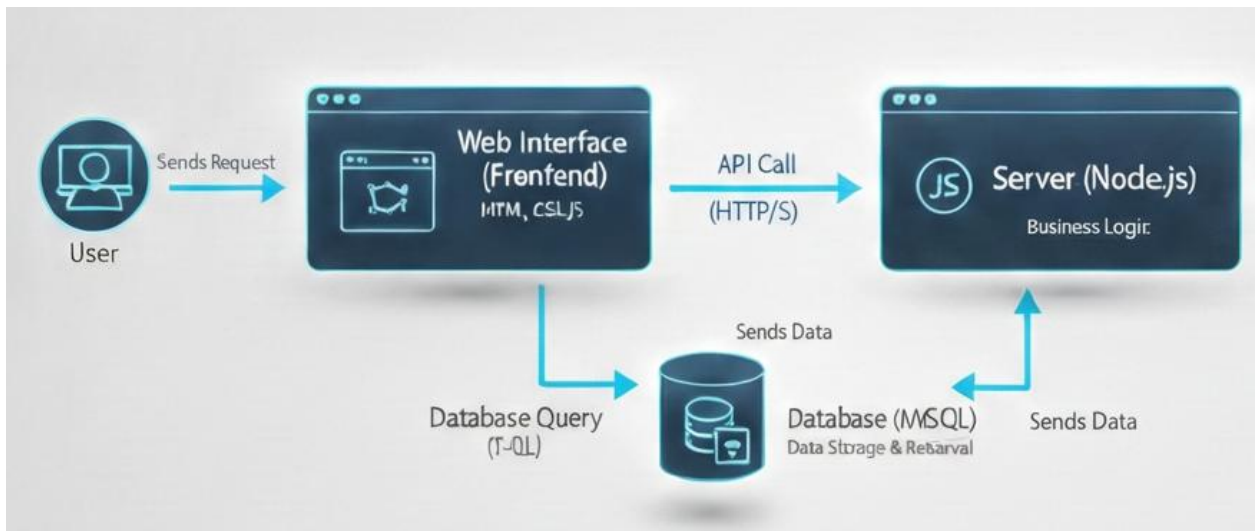


Figure 3.1 Deployment Diagram of Event Management

3.3 Logical Viewpoint

The logical viewpoint describes the static structure of the system in terms of classes and their relationships. It shows key classes such as User, Event, Booking, Organizer, and Admin along with their attributes and associations. A class diagram is used to represent how these classes

interact and depend on each other. This viewpoint helps in understanding the internal design of the system.

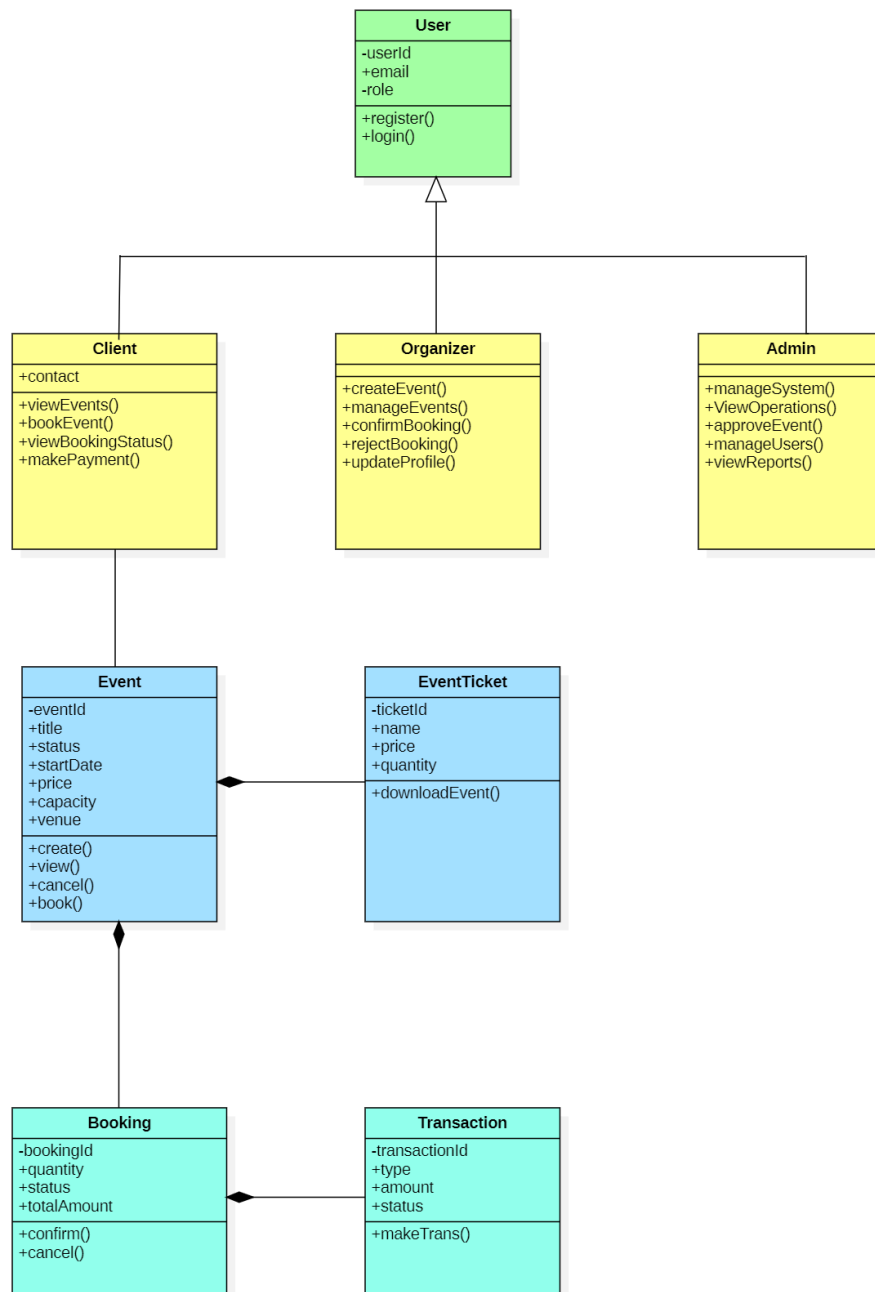


Figure 3.2 Class Diagram of Event Management

3.4 Information Viewpoint

The Information Viewpoint explains how data is stored and managed in the Event Management System. An Entity Relationship Diagram (ERD) represents the database entities, their attributes, and relationships. It shows how users, events, bookings, tickets, and transactions are connected for persistent data storage.

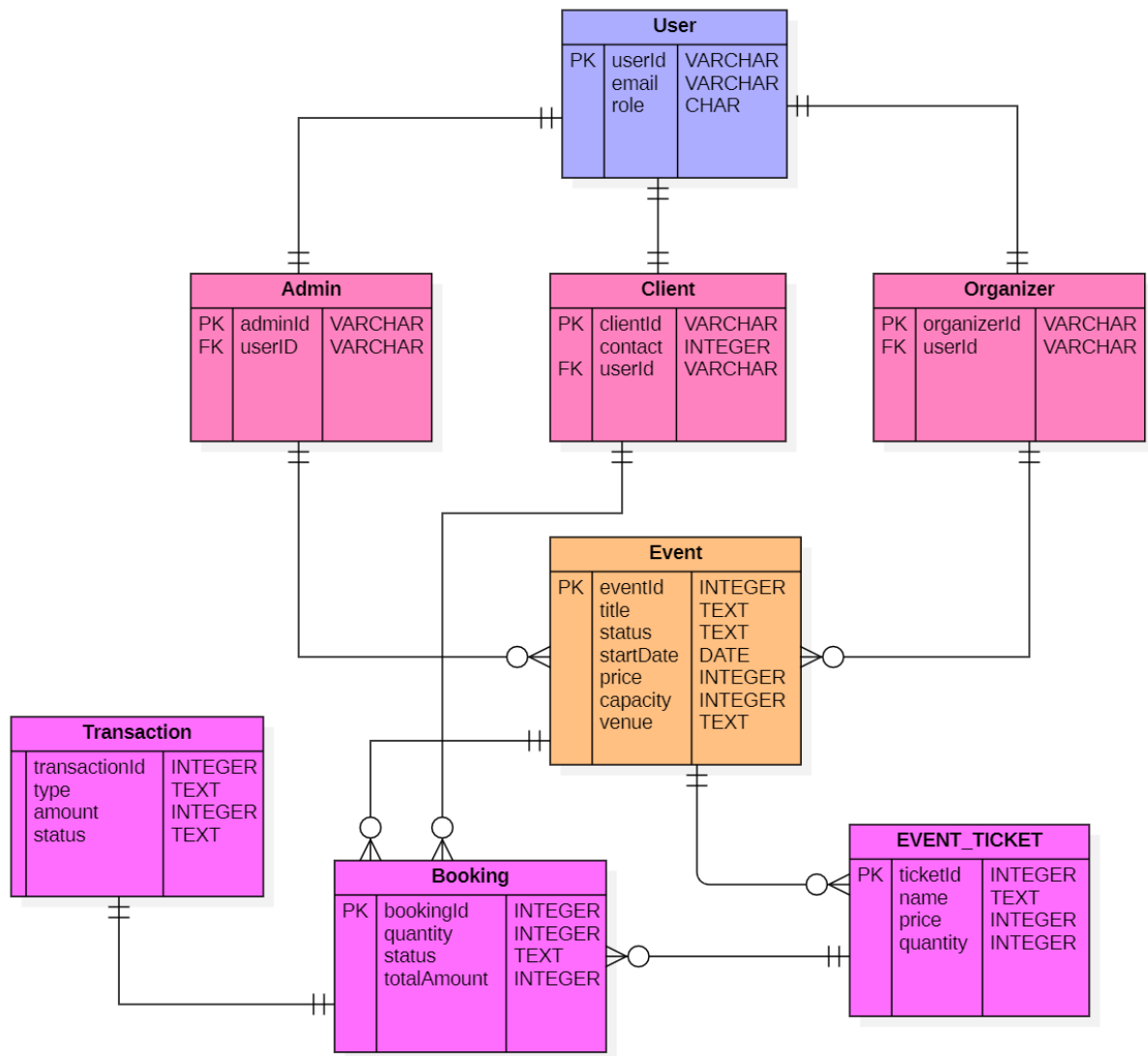


Fig-3.3 ER Diagram of Event Management

3.5 Interaction Viewpoint

The Interaction Viewpoint describes how different actors such as Admin, Organizer, and User

interact with the Event Management System to complete system operations. Sequence diagrams are used to show the flow of messages between actors, the system, and the database for each use case. These diagrams explain how requests are processed step by step, from user action to system respons

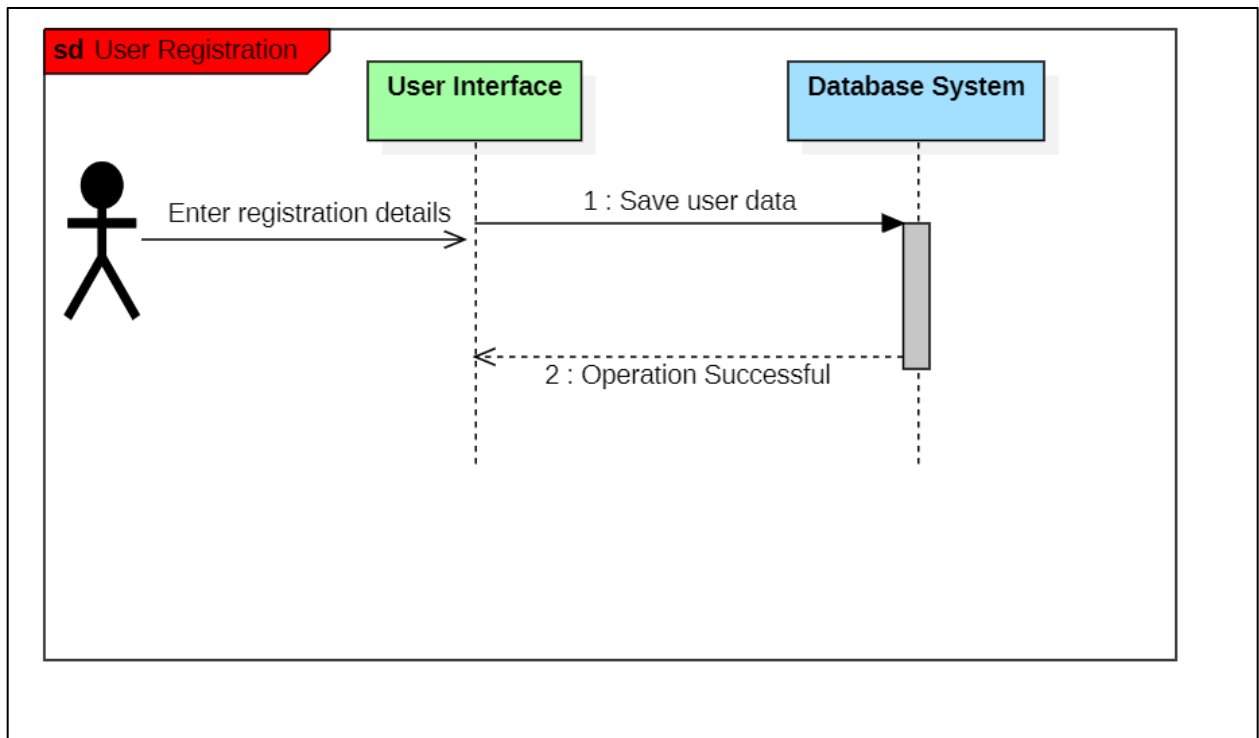


Figure 3.3 Sequence Diagram of Event Management

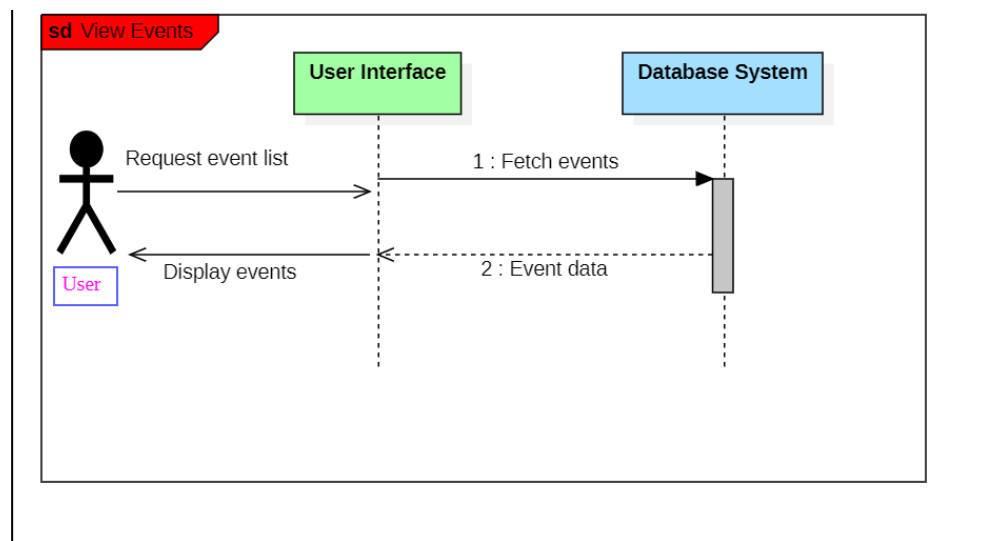


Figure 3.4 Sequence Diagram of Event Management

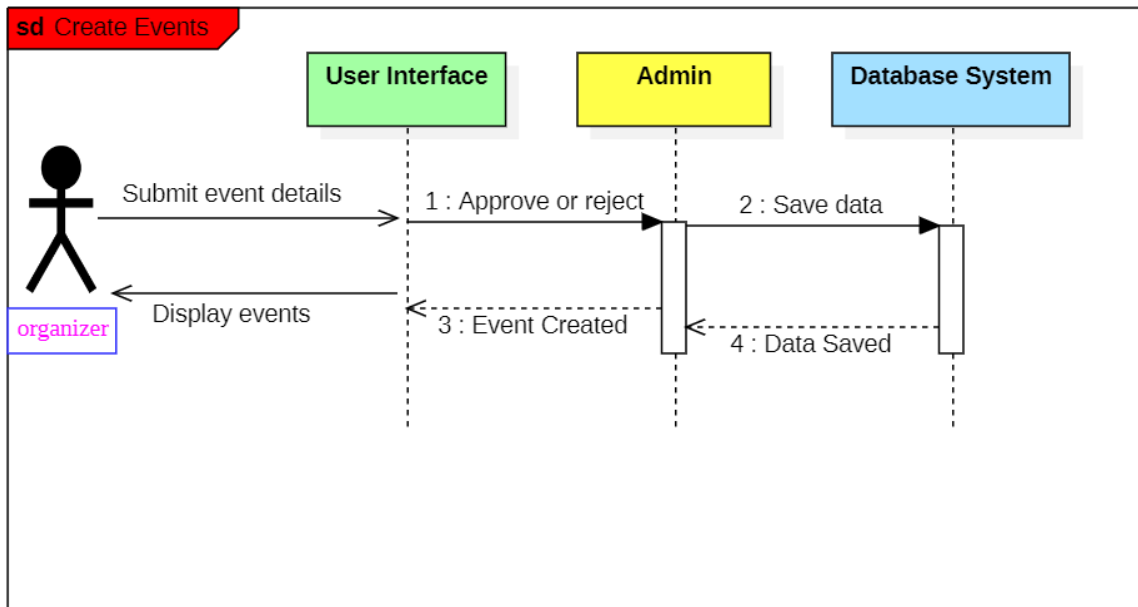


Figure 3.5 Sequence Diagram of Event Management

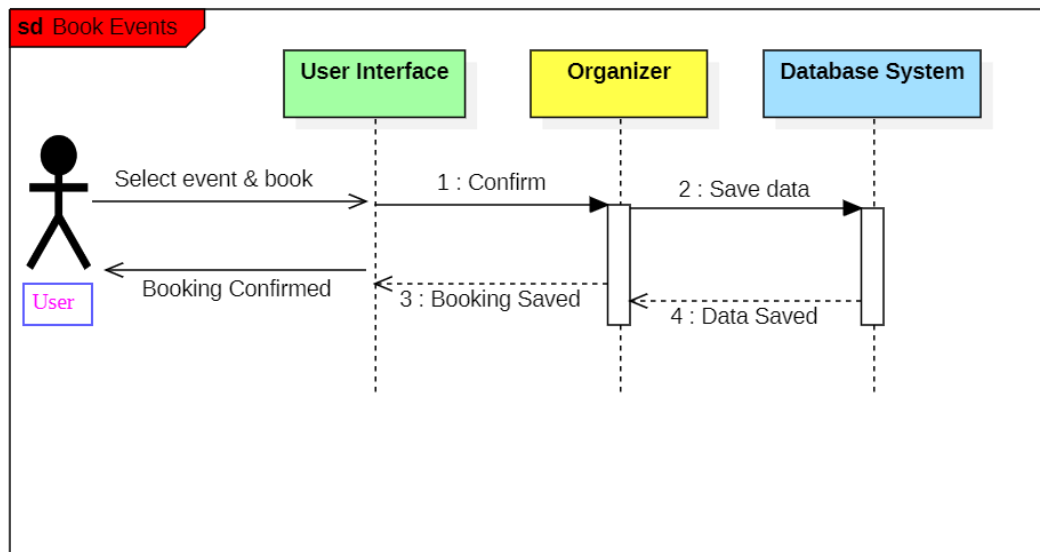


Figure 3.6 Sequence Diagram of Event Management

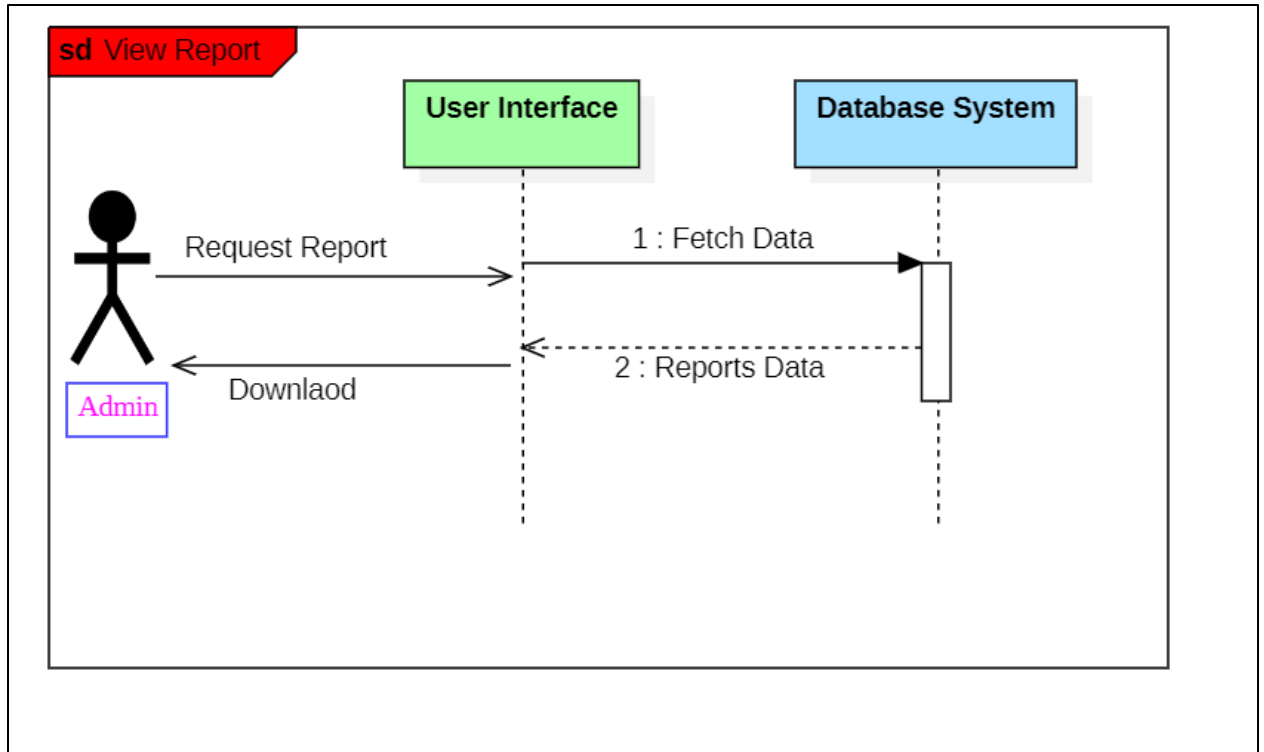


Figure 3.7 Sequence Diagram of Event Management

3.6 State Dynamics Viewpoint

The State Dynamic Viewpoint illustrates how the states of key system entities change over time in the Event Management System. A UML State Machine Diagram is used to represent the lifecycle of an event, from creation and approval to publication, cancellation, or completion. This diagram helps in understanding how different actions trigger state transitions within the system.

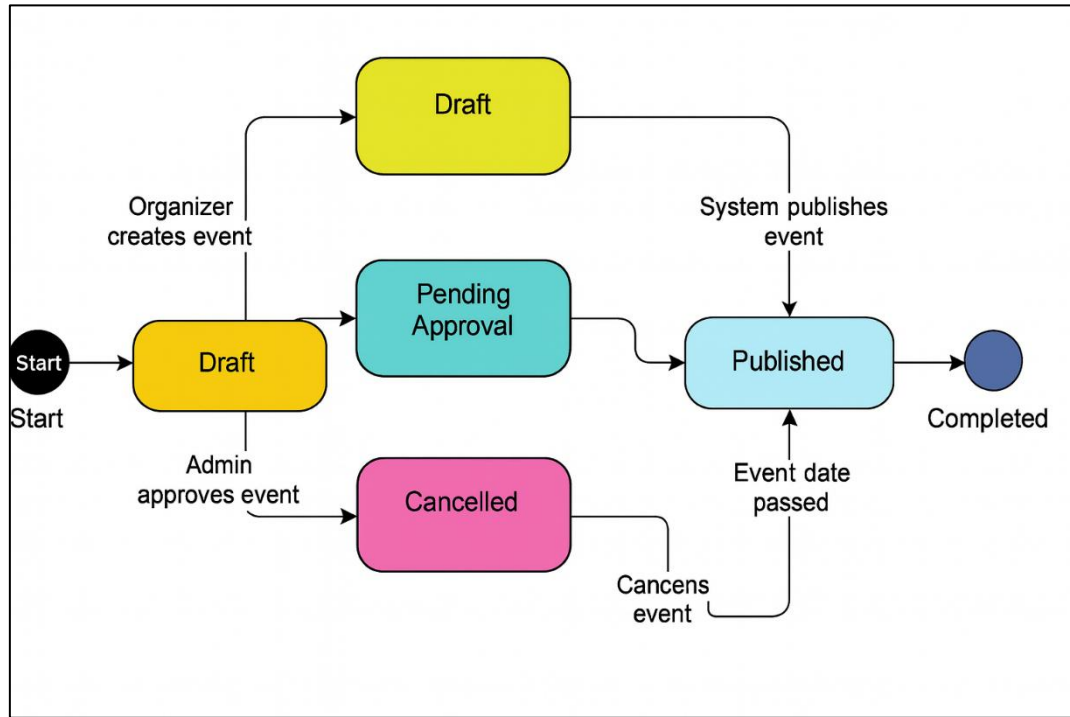


Figure 3.8 State Machine Diagram of Event Management

3.7 Algorithmic Viewpoint

The Algorithmic Viewpoint presents the detailed logic of the Event Management System by describing how system operations are performed internally. It focuses on the behavior of major functions such as user management, event creation and approval, booking, payment, and reporting. Pseudocode is used to clearly explain the step-by-step execution of these operations in a structured and understandable manner.

```
// =====
// EVENT MANAGEMENT SYSTEM - COMPLETE PSEUDOCODE FLOW
// =====

// ----- SYSTEM START -----
START SYSTEM

// ----- USER REGISTRATION -----
INPUT userDetails
IF userDetails are valid THEN
    SAVE userDetails in database
    DISPLAY "Registration Successful"
ELSE
    DISPLAY "Registration Failed"
```

```
ENDIF

// ----- USER LOGIN -----
INPUT email, password
IF credentials match database THEN
    SET userSession = active
    DISPLAY "Login Successful"
ELSE
    DISPLAY "Invalid Credentials"
    EXIT SYSTEM
ENDIF

// ----- ROLE CHECK -----
IF userRole == "Organizer" THEN

    // ----- CREATE EVENT -----
    INPUT eventDetails
    SET eventStatus = "Pending"
    SAVE event in database
    DISPLAY "Event Created and Waiting for Approval"

ENDIF

IF userRole == "Admin" THEN

    // ----- APPROVE OR REJECT EVENT -----
    FETCH pendingEvents
    INPUT adminDecision
    IF adminDecision == "Approve" THEN
        UPDATE eventStatus = "Approved"
        DISPLAY "Event Approved"
    ELSE
        UPDATE eventStatus = "Rejected"
        DISPLAY "Event Rejected"
    ENDIF

ENDIF

IF userRole == "User" THEN

    // ----- VIEW EVENTS -----
    FETCH events WHERE status == "Approved"
    DISPLAY eventList

    // ----- BOOK EVENT -----
```

```
INPUT selectedEvent, selectedTicket, quantity
IF ticketQuantity >= quantity THEN
    CREATE booking
    SET bookingStatus = "Pending Payment"
    DISPLAY "Booking Created"
ELSE
    DISPLAY "Tickets Not Available"
    EXIT SYSTEM
ENDIF

// ----- MAKE PAYMENT -----
INPUT paymentDetails
IF payment successful THEN
    UPDATE bookingStatus = "Confirmed"
    CREATE transaction record
    DISPLAY "Payment Successful"
ELSE
    UPDATE bookingStatus = "Cancelled"
    DISPLAY "Payment Failed"
    EXIT SYSTEM
ENDIF

ENDIF

IF userRole == "Organizer" THEN

    // ----- CONFIRM OR REJECT BOOKING -----
    FETCH bookings
    IF bookingPaymentStatus == "Confirmed" THEN
        CONFIRM booking
        DISPLAY "Booking Confirmed"
    ELSE
        REJECT booking
        DISPLAY "Booking Rejected"
    ENDIF

ENDIF

IF userRole == "Admin" THEN

    // ----- VIEW REPORTS -----
    FETCH eventData, bookingData, transactionData
    GENERATE reports
    DISPLAY reports
```

```
ENDIF

// ----- NOTIFICATION SYSTEM -----
IF any important action occurs THEN
    GENERATE notification
    SEND notification to related user
ENDIF

// ----- SYSTEM END -----
END SYSTEM
```

4 DEVELOPMENT AND TOOLS

4.1 Introduction

This chapter explains the development process of the Event Management System. It describes how the project was planned and implemented by the development team. The tools and technologies used during system development are also discussed. The chapter highlights the roles of team members in different development activities. Finally, it briefly outlines future enhancements for the system.

4.2 Development Plan

This project is developed by a team of four members:

Team Members:

1. Sami Khan
2. Ahmer Abbas
3. Mustafa Riaz
4. Naveed Umer

Phase	Activities	Responsible Members
Requirement Analysis	Understanding problems, defining scope	All Members
System Design	UML diagrams, ERD, architecture design	Sami, Ahmer
Frontend Development	UI design, pages, user interaction	Sami, Mustafa
Backend Development	Business logic, event handling	Sami, Mustafa
Database Development	Database schema and relationships	Ahmer, Naveed
Testing & Debugging	Functional testing, bug fixing	Naveed, Ahmer
Documentation	Final report and diagrams	All Members

Table 4.1 Work Plan division

4.3 Development Tools

The following tools and technologies were used for the development of the Event Management System:

- **Programming Languages:** HTML, CSS, JavaScript

- **Backend Runtime:** Node.js, Express.js
- **Database Management System:** SQL Server Management Studio
- **Integrated Development Environment (IDE):** Visual Studio Code
- **Modeling Tools:** StarUML (for UML, ERD, and diagrams)
- **Version Control System:** Git and GitHub
- **Web Browser:** Google Chrome (for testing and debugging)

4.4 Conclusion and Future Work/Extensions

The Event Management System successfully provides an efficient platform for managing events, bookings, and user interactions. The system supports multiple roles such as admin, organizer, and user, ensuring controlled access and smooth workflow. It improves event handling by automating event approval, booking, and reporting processes, making the overall system reliable and user-friendly.

In the future, the system can be extended by integrating an online payment gateway, developing a mobile application version, and adding real-time notifications. Advanced reporting features, enhanced security mechanisms, and performance optimizations can also be introduced to further improve the system and user experience.

5 QUALITY ASSURANCE

5.1 Introduction

The quality assurance phase ensures the reliability and correctness of the Event Management System through a structured testing approach. Black box testing techniques, including *Equivalence Partitioning*, *Boundary Value Analysis*, *Decision Table Testing*, *State Transition Testing*, and *Use Case Testing*, were used to validate system functionality based on inputs and expected outputs. White box testing techniques, such as *Statement Coverage*, *Branch Coverage*, *Condition Coverage*, *Path Coverage*, and *Loop Testing*, were applied to verify internal logic and control flow of critical modules. Test cases and a requirement traceability matrix were prepared to ensure that all functional requirements were fully tested.

5.2 Traceability Matrix

The Requirement Traceability Matrix maps each functional requirement of the Event Management System with its corresponding test cases. This matrix ensures that all user requirements are properly tested and verified during the quality assurance phase.

TestCases:

1. **TC_1:** Validate user registration with valid inputs
2. **TC_2:** Validate user login with correct credentials
3. **TC_3:** Verify event creation by organizer
4. **TC_4:** Verify event editing functionality
5. **TC_5:** Verify admin event approval
6. **TC_6:** Verify admin event rejection & notification
7. **TC_7:** Verify event listing for users
8. **TC_8:** Verify booking creation
9. **TC_9:** Verify payment processing
10. **TC_10:** Verify report generation

Req ID	Req Description	TC_1	TC_2	TC_3	TC_4	TC_5	TC_6	TC_7	TC_8	TC_9	TC_10	TC
Req_1	User Registration	✓										1
Req_2	User Login		✓									1
Req_3	Create Event (Organizer)			✓								1
Req_4	Edit Event (Organizer)				✓							1
Req_5	Approve/ Reject Event (Admin)					✓	✓					2
Req_6	View Events (User)							✓				1
Req_7	Book Event (User)								✓	✓		2
Req_8	Make Payment									✓		1
Req_9	View Reports (Admin)										✓	1

Table 5.1 Traceability Matrix

5.3 Test Plan

The test plan defines the testing mechanism used to verify the functionality of the Event Management System. Each functional requirement was tested using well-defined test cases to ensure correct system behavior. The following tables describe the test cases conducted during the quality assurance phase.

Field	Description
Test ID	TC-01
Test Name	User Registration
Date of Test	14/12/2025
Name of Application	Event Management System
Description	System allows a new user to register by providing valid details
Input	Enter valid email and password, click register
Expected Output	User account created successfully
Actual Output	User account created successfully
Test Role (Actor)	User
Test Verified By	Team Members

Table 5.2 Test Case User Registration

Field	Description
Test ID	TC-02
Test Name	User Login
Date of Test	10/12/2025
Name of Application	Event Management System
Description	System allows registered users to log in
Input	Enter valid login credentials
Expected Output	User logged in successfully
Actual Output	User logged in successfully
Test Role (Actor)	User
Test Verified By	Team Member

Table 5.3 Test Case User Login

Field	Description
Test ID	TC-03
Test Name	Create Event
Date of Test	09/12/2025
Name of Application	Event Management System
Description	Organizer creates a new event
Input	Enter event details and submit
Expected Output	Event created with pending approval status
Actual Output	Event created successfully
Test Role (Actor)	Organizer
Test Verified By	Team Member

Table 5.4 Test Case Create Event

Field	Description
Test ID	TC-04
Test Name	Approve Event
Date of Test	05/12/2025
Name of Application	Event Management System
Description	Admin approves an event created by organizer
Input	Click approve event
Expected Output	Event status updated to approved
Actual Output	Event approved successfully
Test Role (Actor)	Admin
Test Verified By	Team Member

Table 5.5 Test Case Approve Event

Field	Description
Test ID	TC-05
Test Name	Book Event
Date of Test	14/12/2025
Name of Application	Event Management System
Description	User books an approved event

Input	Select event and confirm booking
Expected Output	Booking created successfully
Actual Output	Booking created successfully
Test Role (Actor)	User
Test Verified By	Team Member

Table 5.6 Test Case Booking Event

Field	Description
Test ID	TC-06
Test Name	Make Payment
Date of Test	12/12/2025
Name of Application	Event Management System
Description	System allows users to make payment for an event booking
Input	Enter valid payment details and submit
Expected Output	Payment processed successfully and booking confirmed
Actual Output	Payment processed successfully and booking confirmed
Test Role (Actor)	User
Test Verified By	Team Member

Table 5.7 Test Case Make Payment

Field	Description
Test ID	TC-07
Test Name	View Booking Status
Date of Test	12/12/2025
Name of Application	Event Management System
Description	System allows users to view booking status
Input	Open booking history
Expected Output	Booking status displayed correctly
Actual Output	Booking status displayed correctly
Test Role (Actor)	User
Test Verified By	Team Member

Table 5.8 Test Case View Booking Status

Field	Description
Test ID	TC-08
Test Name	Edit Event
Date of Test	14/12/2025
Name of Application	Event Management System
Description	Organizer edits an existing event
Input	Update event details and submit
Expected Output	Event details updated successfully
Actual Output	Event details updated successfully
Test Role (Actor)	Organizer
Test Verified By	Team Member

Table 5.9 Test Case Edit Event

Field	Description
Test ID	TC-09
Test Name	View Reports
Date of Test	10/12/2025
Name of Application	Event Management System
Description	Admin views system reports
Input	Select report option from dashboard
Expected Output	Reports displayed successfully
Actual Output	Reports displayed successfully
Test Role (Actor)	Admin
Test Verified By	Team Member

Table 5.10 Test Case View Reports

6 USER MANUAL

6.1 Introduction

This chapter explains how to install and use the Event Management System. It describes the hardware and software requirements needed to run the system. Step-by-step installation instructions are provided for users. The chapter also includes an operating manual explaining how admin, organizer, and users can access system functionalities.

6.2 Hardware/Software Requirements for the System

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Hard Disk: Minimum 10 GB free space
- Display: 1366×768 resolution or higher
- Internet Connection (for system access)

Software Requirements

- Operating System: Windows 10 / Windows 11
- Web Browser: Google Chrome / Microsoft Edge
- Backend Runtime: Node.js
- Database Server: SQL Server
- IDE (for development): Visual Studio Code

6.3 Installation guide for Application

The following steps are used to install and run the Event Management System:

- Install Node.js on the system
- Install SQL Server / MySQL and create the required database
- Open the project folder in Visual Studio Code
- Configure database connection settings
- Run the backend server using terminal command
- Open the application in a web browser using localhost URL

6.4 Operating Manual

This section provides step-by-step guidelines to operate the Event Management System. It explains how different users can access and use system functionalities based on their roles. Screenshots of the application interfaces can be added alongside each step to make the process easier to understand.

6.4.1 Starting the Application

- Open a web browser (Google Chrome recommended).
- Enter the application URL (localhost) in the address bar.
- The home page of the Event Management System is displayed.

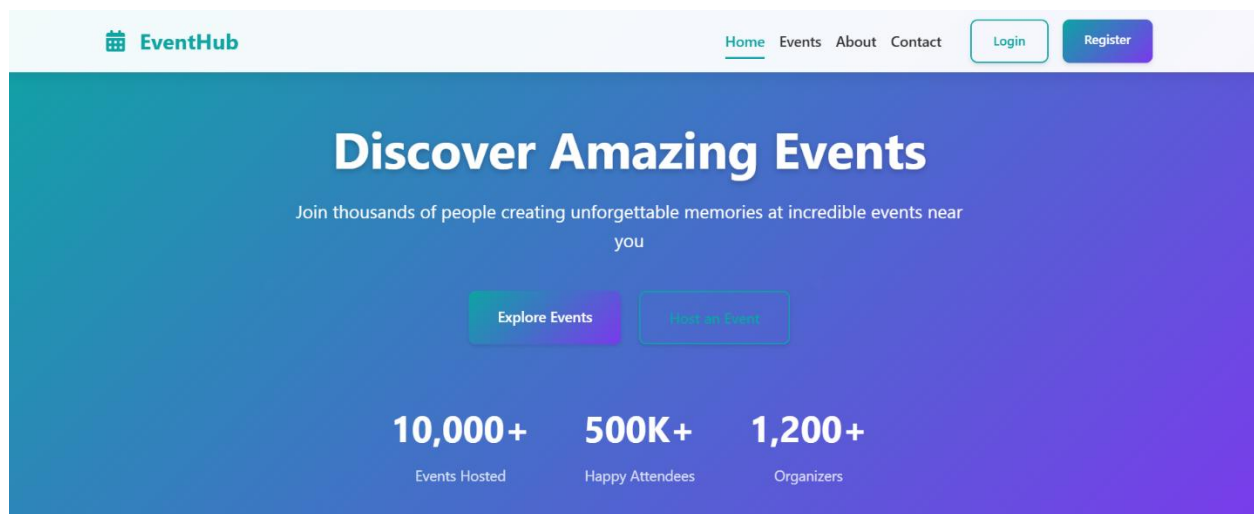
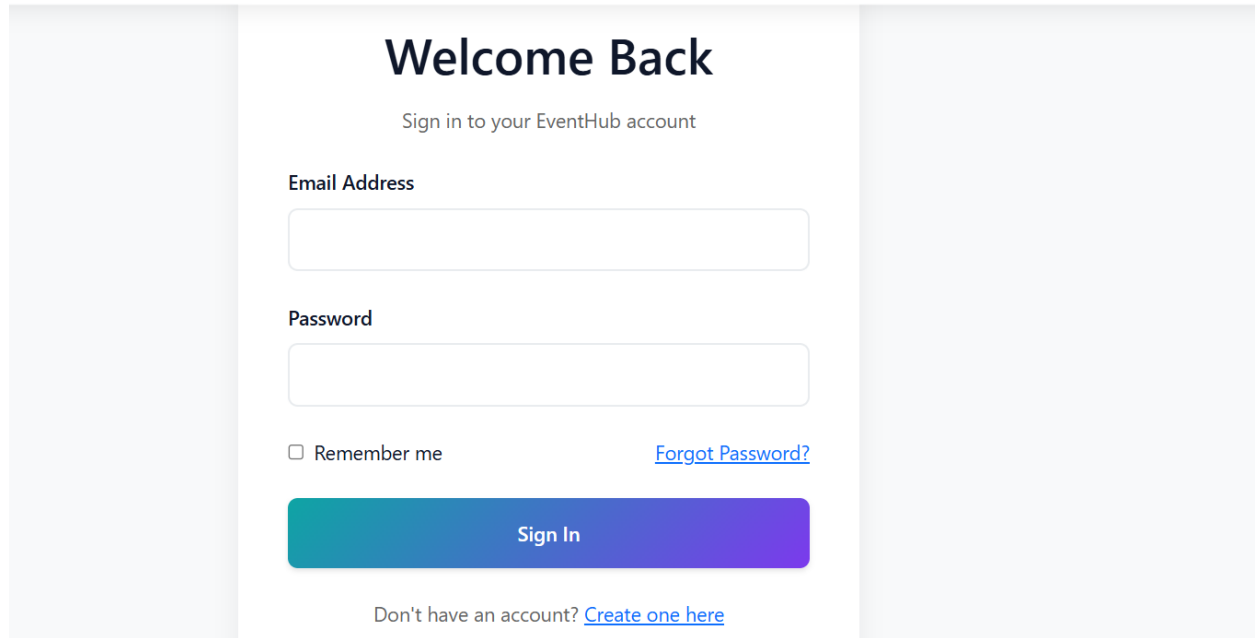


Figure 6.1 Home Page of Event Management

Login:

1. Click on the Login button.
2. Enter registered email and password.
3. Click Login to access the system.



Welcome Back

Sign in to your EventHub account

Email Address

Password

☐ Remember me [Forgot Password?](#)

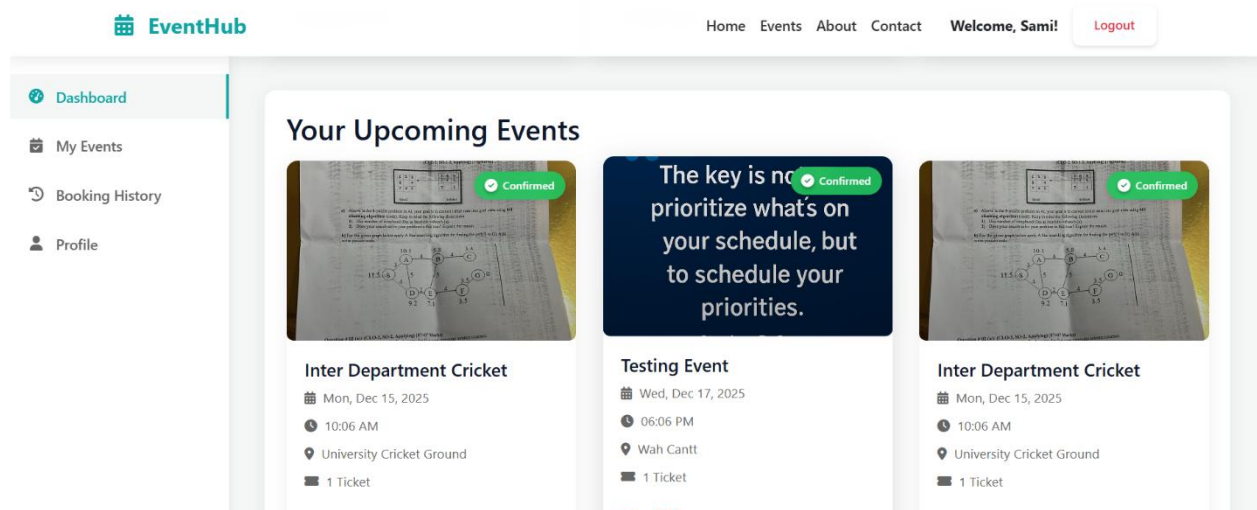
Sign In

Don't have an account? [Create one here](#)

Figure 6.2 Login Page of Event Management

6.4.2 User Operations:

- Login as a user.
- View the list of approved events on the dashboard.
- Select an event to view details.
- Click Book Event to make a booking.
- Complete payment to confirm the booking.
- View booking status and event details from the user dashboard.



EventHub Home Events About Contact Welcome, Sami! Logout

Dashboard My Events Booking History Profile

Your Upcoming Events

Inter Department Cricket

Mon, Dec 15, 2025

10:06 AM

University Cricket Ground

1 Ticket

Confirmed

Testing Event

Wed, Dec 17, 2025

06:06 PM

Wah Cantt

1 Ticket

Confirmed

Inter Department Cricket

Mon, Dec 15, 2025

10:06 AM

University Cricket Ground

1 Ticket

Confirmed

Figure 6.3 User Dashboard of Event Management

6.4.3 Organizer Operations:

- Login as an organizer.
- Click Create Event to add a new event.
- Enter event details and submit the form.
- View and manage created events from the organizer dashboard.
- View bookings related to events.
- Confirm or reject bookings based on payment status.

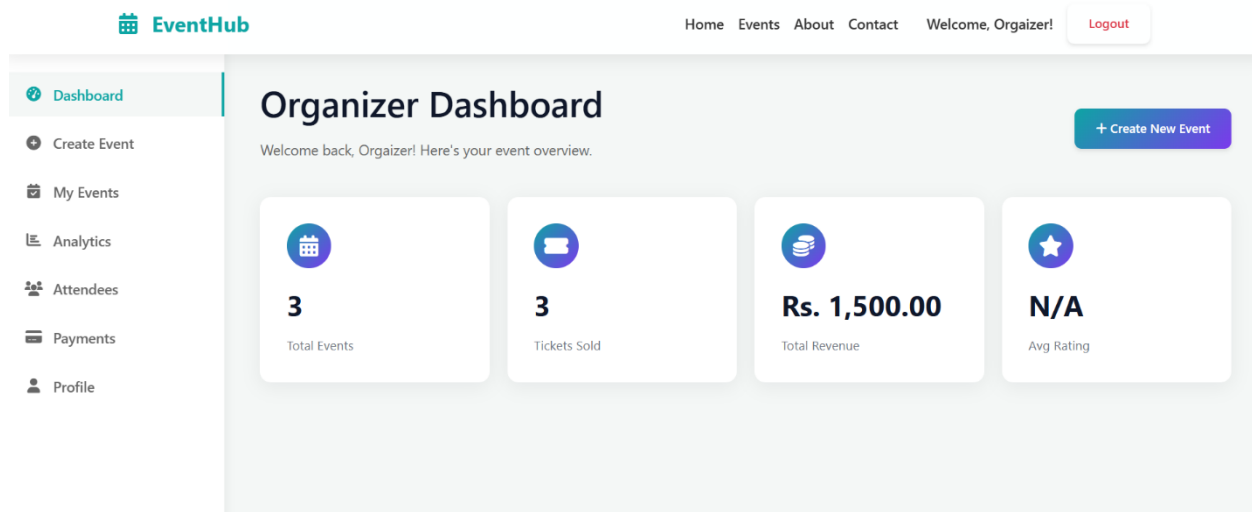


Figure 6.4 Organizer Dashboard of Event Management

6.4.4 Admin Dashboard:

- Login using admin credentials.
- View all events created by organizers.
- Approve or reject pending events.
- Manage users and organizers.
- View reports related to events, bookings, and payments.

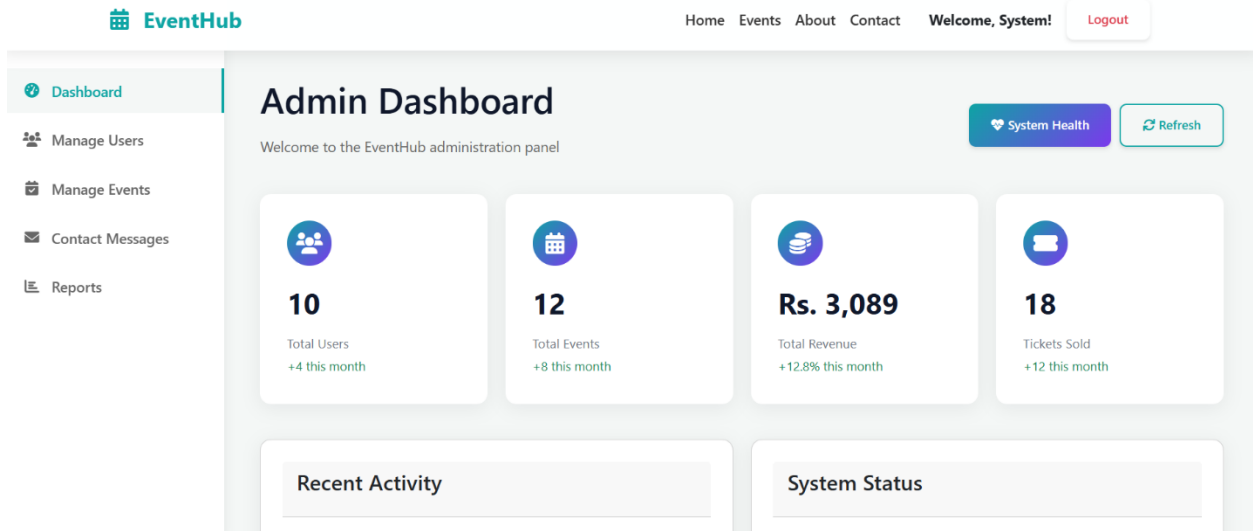


Figure 6.5 Admin Dashboard of Event Management